Чек-лист по тестированию HTML-верстки

Группы основных проверяемых параметров:

Оглавление

Чек-лист по тестированию HTML-верстки 1

1. Соответствие макету 2

2. Работа в разных окружениях 2

3. Проверка на разных разрешениях экрана (проверка десктопной и адаптивных версий) 3

4. HTML 5

5. Javascript 6

6. CSS 8

7. Фавиконки 9

8. Шрифты 10

9. Навигация 11

10. Заголовки 12

11. Контент 12

12. Изображения 13

13. Ссылки и кнопки 15

14. Футер 16

15. Формы 17

1. Соответствие макету

- **Расхождение макета и верстки в пикселях**

  Мы используем для этого браузерный плагин PerfectPixel и сайт I love adaptive. Существует также множество других программ и расширений для проверки соответствия верстки и макета.

- **Корректный вывод элементов интерфейса в векторном формате**

  В идеале, все изображения в элементах интерфейса должны быть в формате svg. Изображения в векторном формате корректно масштабируются на любых разрешениях экранов и типов мониторов.

- **Поддержка retina-мониторов**

  Реальный размер растровых изображений интерфейса должен быть как минимум в 2 раза больше, чтобы корректно смотреться на экранах с повышенной четкостью.

- **Наличие элементов, выбивающихся из цветовой гаммы сайта**

  В крупных проектах очень легко не заметить расхождения каких-либо элементов по цвету с остальным сайтом. Какие-то элементы, например, вывод ошибок, могут быть не отрисованы в макете, и их корректное отображение ложится на плечи верстальщиков.

- **Наличие элементов с малой контрастностью**

  Есть несколько инструментов, позволяющих проверить данный параметр, например Google Lighthouse.

2. Работа в разных окружениях

- **Кроссбраузерность**

  На этапе разработки проверяем сайт в браузере Chrome. На этапе тестирования необходимо проверить сайт в браузерах, поддержка которых указана в ТЗ проекта.

- **Корректная работа на разных устройствах**

  Для тестирования на реальных устройствах мы используем сервис lambdatest.com или BrowserStack.

- **Корректная работа на разных операционных системах**

  Необходимо проверить на Windows и MacOS.

- **Корректная работа сайта при различных настройках местоположения пользователя, часового пояса и времени**

  Например, отображение попапа при определении города.

- **Корректная работа с разной скоростью интернета**

Некоторые асинхронные ресурсы на сайте могут не успевать загружаться перед другими ресурсами при медленной скорости интернета. Настроить разную скорость интернета можно, например, в инструментах разработчика Chrome.

- **Корректная работа при включенном расширением AdBlock в браузере**
- **Наличие мета-тега <meta http-equiv="X-UA-Compatible" content="IE=edge">**

  Этот тег необходим для корректной работы в Internet Explorer. По данному тегу браузер определяет, с помощью какой версии движка рендерить сайт.

- **Корректное отображение кнопок, полей ввода, выпадающих списков и радиокнопок на разных устройствах**

  Нужно уделять этому внимание, так как устройства могут применять свои стили.

- **Корректное отображение сайта с административной панелью CMS**

  Административная панель CMS обычно отображается в верхней части сайта и может сдвинуть контент. Стили и скрипты должны учитывать это.

- **Корректная настройка встраиваемых карт для тач-устройств**

  На тач-устройствах в картах Яндекс и Google желательно отключать zoom карты при скролле, так как пользователь не сможет проскроллить страницу.

- **Корректное фиксирование хедера при прокрутке**

  Часто на сайтах хедер слишком резко переходит из статичного состояния в закрепляемое, из-за этого при прокрутке происходит прыжок. Чтобы этого не происходило, хедер должен закрепляться плавно, вслед за прокруткой.

3. Проверка на разных разрешениях экрана (проверка десктопной и адаптивных версий)

- **Корректное отображение на всех возможных размерах окна браузера**

  Мы проверяем базовое соответствие макету в iloveadaptive.com или с помощью инструментов разработчика Chrome.

- **Область нажатия кнопок**

  Нужно предусматривать, что на тач-устройствах на кнопки будут нажимать пальцами, поэтому все интерактивные элементы должны быть оптимального размера, чтобы можно было на них нажать, не задевая другие элементы.

- **Отображение страницы при масштабировании в десктопных браузерах**

  Сетка страницы не должна ломаться при масштабировании: элементы интерфейса не должны «залезать» на другие элементы.

- **Корректное отображение страницы при масштабировании в мобильных браузерах**

- **Наличие мета-тега viewport**

Данный мета-тег отвечает за корректное отображение сайта на различных разрешениях экрана. Пример оптимального заполнения данного мета-тега: <meta name="viewport" content="width=device-width, initial-scale=1">.

- **Корректная прокрутка страницы при открытых попапах**

  Часто на мобильных телефонах при открытии попапа, прокрутка основной страницы не блокируется, что может привести к залипанию прокрутки. На десктопе этого не видно, поэтому подобное поведение необходимо отдельно проверять на реальных мобильных устройствах.

- **Корректное отображение плавающих (закрепляемых за прокруткой) элементов**

  Плавающие кнопки, например кнопки открытия чата, должны корректно отображаться на всех разрешениях экрана и не закрывать собой контент страницы.

- **Использование чрезмерно большого количества различных брейкпоинтов для стилей**

  Необходимо использовать 5-7 различных брейкпоинтов для изменения стилей на разных разрешениях экрана. Если их используется больше, то сайт становится сложнее разрабатывать, повышается вероятность не заметить какое-либо изменение и пропустить баг.

  Мы используем такой набор брейкпоинтов: 414px - 576px - 768px - 992px - 1200px - 1366px.

- **Смешивание различных брейкпоинтов для стилей**

  Основная часть брейкпоинтов должна «смотреть» в одну сторону: необходимо использовать либо брейкпоинты min-width, либо max-width. Часто на сайтах используется оба вида брейкпоинтов, что приводит к багам на определенных разрешениях экрана. Кроме того, для min-width и max-width должны использоваться разные брейкпоинты, например, если для min-width используется ширина 768px, то для max-width нужно использовать ширину 767px.

- **Слишком узкие блоки на маленьких разрешениях экрана**

  На разрешениях экрана меньше 500px для контента остаётся совсем мало места, особенно, если есть несколько блоков с отступами, вложенных друг в друга. Если есть два таких блока и у каждого отступ 15px, то с обеих сторон экрана «съедается» 60px. Для таких случаев рекомендуется прижимать эти блоки к краю экрана, чтобы отступ до контента составлял с каждой стороны 15-20px.

- **Наличие сломанной вёрстки при изменении размеров экрана**

  Часто при изменении размеров экрана должны отрабатывать js-скрипты, отвечающие за корректное отображение верстки при текущем разрешении. Нужно отслеживать, чтобы скрипты отрабатывали не один раз на странице, а каждый раз при изменении размеров экрана.

- **Некорректная верстка на мобильных устройствах при показе/скрытии адресной строки**

  Высота в CSS на мобильных телефонах вычисляется с учетом адресной строки, поэтому, если необходимо, чтобы высота блока была на всю высоту экрана телефона, лучше это делать с помощью скриптов.

4. HTML

- **Валидность HTML**

  Незакрытые теги, некорректная вложенность тегов, невалидные атрибуты, синтаксические ошибки, устаревшие технологии и т.п. Для проверки используется валидатор W3C.

- **Наличие корректного DOCTYPE**

  Данный тег указывает на версию используемого HTML на странице, поэтому указывать его обязательно нужно.

- **Наличие корректной кодировки**
- **Наличие тега title и мета-тегов для SEO**

  Например, мета-теги с name="keywords" и name="description".

- **Наличие атрибута lang у тега html**

  Например, lang="ru-RU". Данный атрибут необходим для правильного определения языка страницы поисковыми системами. Кроме того, этот атрибут помогает плагинам-переводчикам понять, предлагать ли перевод страницы.

- **Повторяющиеся или некорректные атрибуты id**

  Атрибуты id должны начинаться с буквы, а не с числа.

- **Наличие пустых и ненужных тегов**

  Браузеру приходится обрабатывать каждый тег при загрузке страницы, поэтому уменьшение количества тегов приведет к ускорению работы сайта.

- **Наличие объёмных комментариев в коде**

  Во-первых, такие комментарии увеличивают объем страницы, а во-вторых, подобные комментарии видны всем, поэтому это может быть небезопасно. Вместо HTML-комментариев нужно использовать комментарии в PHP.

5. Javascript

- **Не должно быть ошибок javascript при работе с сайтом**

  Один из самых важных пунктов. Каждая javascript-ошибка может повлечь за собой неработающий или некорректно работающий функционал.

- **Скрипты должны быть объединены в один файл и минифицированы**

  Уменьшает размер файлов javascript и ускоряет парсинг этих файлов браузером. Также позволяет браузеру делать меньше запросов к серверу. Многие CMS самостоятельно умеют делать подобное, но стоит подробнее ознакомится с их особенностями. Например, Битрикс умеет объединять файлы скриптов в один файл, перемещать их в конец страницы и подключать минифицированные версии файлов (с суффиксом .min). Но битрикс не умеет самостоятельно делать минифицированные версии файлов. Мы минифицируем файлы скриптов с помощью Terser.

- **Файл скриптов должен подключаться внизу страницы**

  Таким образом, вначале страница прогрузится визуально, и пользователь уже сможет взаимодействовать с ней, а потом будет загружаться дополнительный функционал страницы.

- **Использование кода из неподходящей версии javascript**

  Не весь код, написанный на javascript, поддерживается во всех браузерах, поэтому необходимо следить, чтобы все участки кода корректно работали во всех браузерах. Особенно эта проблема относится к браузеру Internet Explorer. Многие плагины напрямую перестали поддерживать работу в этом браузере, и для их корректной работы необходимо ставить полифилы, т.е. небольшие скрипты, позволяющие более новому синтаксису javascript работать в неподдерживаемых браузерах. Совместимость javascript-кода можно проверить на специальном сайте.

- **Сторонние скрипты желательно должны иметь атрибуты async и defer**

  Речь о скриптах для сбора метрики, либо о сторонних скриптах для функционала, который не критичен при загрузке всей страницы, например Яндекс.карты. Наличие этих атрибутов позволяет браузеру лучше определять приоритет при загрузке ресурсов страницы.

- **Наличие устаревших javascript-плагинов**

  Обычно, устаревшие плагины больше весят, медленнее работают, содержат код для устаревших браузеров и небезопасны.

- **Корректное подключение сторонних библиотек**

  Сторонние скрипты необходимо подключать локально, только таким образом можно быть уверенным в том, что все скрипты загрузятся, и загрузятся в одно время.

- **Некорректная работа плагинов**

  Если сторонний плагин популярен и проверен временем, это не значит, что в нём нет багов. Стили сторонних плагинов не должны выделяться на фоне остального сайта, а в работе с данным плагином не должно возникать javascript-ошибок. В данном случае можно или

исправить код плагина самому, или найти какой-нибудь форк, где этот баг исправлен, или найти другой подобный плагин, где этой ошибки не будет.

- **Наличие непереведенного текста в плагинах**

  Большинство плагинов имеют встроенные возможности для локализации, поэтому, при добавлении нового плагина нужно не забывать переводить его текстовые параметры.

- **Адекватное отображение сайта при выключенном javascript**

  Необходимо, чтобы сайт не «разваливался» при выключенном или еще не загруженном javascript.

- **Наличие кода на «боевом» сайте, предназначенного для разработки на тестовом сервере**

  Это могут быть различные тестовые функции, вызовы console.log, или вывод лишней информации, не предназначенной для обычных пользователей.

- **Корректная инициализация контентных слайдеров**

  При инициализации контентных слайдеров необходимо проверять, достаточно ли внутри них элементов для прокрутки, а также, не ломается ли высота элементов внутри слайдера.

6. CSS

- **Валидация CSS**

  Поиск синтаксических ошибок в стилях, поиск некорректных или устаревших правил. Валидация стилей производится с помощью W3C валидатора CSS.

- **Файлы стилей должны быть корректно подключены**

  Файлы стилей должны подключаться в head и загружаться перед всем видимым контентом.

- **Стили должны быть объединены в один файл и минифицированы**

  По аналогии с файлами скриптов, это позволит уменьшить размер файлов стилей.

- **Наличие в файлах стилей лишних правил для не поддерживаемых браузеров**

  Многие стили пишутся только для корректной работы на старых браузерах, и периодически такие стили кочуют из проекта в проект, хотя браузером или устройством, для которого они создавались, больше никто не пользуется.

- **Использование контентных тегов для стилизации**

  К таким тегам относятся br, b, strong, i, em, p, s. Данные теги необходимо использовать только в контенте, но не в стилизации интерфейса.

- **Стилизация элементов по атрибутам name или id**

  Эти атрибуты используются программистами и могут быть изменены ими. Для стилизации необходимо использовать классы или data-атрибуты.

- **Использование одинаковых цветов, скруглений, отступов, размеров шрифтов**

  Если на сайте используется очень большое количество разных стилевых элементов с разными размерами, то сайт начинает выглядеть неаккуратно. Подобные вещи нужно сохранять в виде переменных и использовать во всем проекте.

- **Использование unicode-символов в файлах стилей и скриптов**

  Если сайт будет работать в кодировке win-1251, или другой кодировке, отличающейся от UTF-8, то эти символы будут отображаться некорректно.

- **Артефакты, возникающие при использовании стилей transform**

  Такое поведение часто встречается в браузерах, когда несколько трансформируемых элементов находятся рядом или имеют нестандартные свойства.

- **Дергающиеся и некорректно работающие анимации**

  Такое может происходить в основном из-за большого количества анимаций на странице или из-за большого веса страницы. Даже если анимация сделана корректно, она может отображаться неровно.

- **Разные стили плавности анимации**

Чтобы сайт смотрелся аккуратнее, все стили transition должны быть с одинаковой длительностью и плавностью. Если на сайте используется transition, то он должен быть у всех анимированных правил.

- **Корректное отображение сайта с включенным режимом редактирования в CMS**

  Например, в CMS Bitrix включение режима редактирования добавляет на сайт дополнительные оборачивающие теги, что может поломать вёрстку сайта, особенно, если он сверстан с использованием flex.

- **Адекватные отступы между блоками контента**

  Часто отступы выглядят хорошо на одном разрешении экрана, но на другом становятся или слишком большими, или слишком маленькими. Нужно адаптировать отступы на разных разрешениях экрана. Требуется обсуждение с дизайнером.

- **Слишком резкая граница для overflow: hidden**

  Такое может быть заметно, если внутри блока с ограничением по видимости контента, находится элемент с тенью и тень обрезается.

- **Наличие горизонтальной прокрутки**

  Часто некоторые элементы за счет отступов начинают немного вылезать за пределы экрана, за счет чего появляется небольшая горизонтальная прокрутка. Особенно часто это заметно на мобильных телефонах.

- **Большой разброс z-index**

  Значения z-index в стилях сайта должны иметь адекватные значения, например от 0 до 100. При корректной верстке должно быть достаточно 5-10 разных значений. Значения с огромными цифрами (например, 100000) ничего не значат, z-index работает по-другому.

7. Фавиконки

- **Наличие favicon.ico и фавиконки больших размеров**

  В настоящее время браузеры умеют показывать иконку сайта в высоком разрешении, поэтому стоит загружать favicon в размере 32x32px.

- **Корректный список подключаемых фавиконок**

  Минимальный набор иконок должен составлять favicon.ico (размер 32x32px) для отображения фавиконки в старых браузерах, favicon.svg для отображения фавиконки в новых браузерах, apple-touch-icon-180x180.png для отображения сайта в мобильных телефонах iPhone. В манифесте должы быть указаны пути к фавиконкам с размерами 192x192px и 512x512px для устройств Andriod.

- **Наличие manifest.json или manifest.webmanifest**

  В этом файле содержится информация, как отображать иконку сайта на разных устройствах, т.к. файл favicon.ico не всегда имеет подходящий размер и формат.

8. Шрифты

- **Наличие малоиспользуемых или подключение неиспользуемых на сайте шрифтов**

  Шрифты могут весить довольно много, и, кроме того, браузер затрачивает дополнительное время на подключение шрифта, поэтому лучше не подключать большое количество разных шрифтов на сайт, а обойтись двумя-тремя видами шрифтов.

- **Правильное подключение шрифтов**

  Необходимо подключать шрифты в современных форматах (woff2 или woff), они меньше весят и быстрее подгружаются.

- **Подключение шрифтов только из локальных источников**

  Не рекомендуется подключать шрифты с помощью ссылки на сторонний ресурс, т.к. в таком случае скорость загрузки сайта становится зависимой от скорости загрузки стороннего сайта, не считая тех случаев, если сторонний сайт становится вообще не доступен.

- **Предзагрузка шрифтов**

  Рекомендуется подключать шрифты до загрузки стилей, чтобы, когда стили прогрузятся, шрифты сразу же отображались корректно.

- **Наличие правила font-display для подключаемых шрифтов**

  Данное правило указывает на то, как текст должен вести себя, пока шрифт загружается. Корректная установка этого правила позволит тексту отображаться раньше загрузки шрифта.

- **Наличие fallback-шрифтов**

  Резервные шрифты необходимо указывать на случай, если основной шрифт не удалось загрузить.

- **Указание наличия или отсутствия засечек при использовании шрифтов**

  В стилях сайта у всех правил для шрифтов должен быть быть указан serif или sans-serif. При невозможности загрузки шрифта, браузер пытается загрузить похожий шрифт, и указание на наличие засечек у шрифта помогает ему точнее найти подобный шрифт.

9. Навигация

- **Битые ссылки**

  Наличие битых ссылок может критически повлиять на функционал сайта, поэтому этот пункт крайне важен. Проверять на наличие битых ссылок необходимо с помощью специальных инструментов, например, с помощью программы Screaming Frog или бесплатной программы Xenu 2.

- **Ссылки, которые ведут на текущую страницу (на главной странице верхний логотип сайта не должен быть ссылкой)**

  Считается, что это может негативно повлиять на представление сайта для поисковых систем.

- **Корректная верстка меню**

  Меню должны быть реализованы через тег nav, внутри которого находятся вложенные ul.

- **Корректная верстка «хлебных крошек»**

  «Хлебные крошки» должны быть оформлены как меню, при этом последний элемент (текущая страница), не должен быть ссылкой. В идеале, хлебные крошки должны содержать микроразметку.

- **Корректное отображение меню при различном количестве пунктов меню**

  Часто на макетах сайта дизайнер красиво и равномерно отображает пункты, но на рабочем сайте количество пунктов меню может быть другим. Необходимо проверять различные количества пунктов в меню в адекватных пределах.

- **Наличие стилей для индикации текущего элемента в навигации и неактивных элементов**

  Например, в слайдерах, если пользователь перешел к последнему слайду, то кнопка «Далее» должна становится неактивной.

10. Заголовки

- **Наличие одного тега h1 на странице**

  На каждой странице должен быть обязательно ровно один тег h1.

- **Семантичность заголовков (должны идти по порядку)**

  Не должны более мелкие (например h3, h4) заголовки идти в основном контенте перед более крупными заголовками (h1, h2).

- **Правильно настроенные стили и соотношение размеров заголовков**

  Заголовки по умолчанию должны выделяться на фоне остального текста, а также размеры заголовков должны соответствовать их уровню (например H1 должен быть самым крупным, h2 поменьше и т.д.).

- **Заголовки должны использоваться только в контентной части, например, заголовки страницы**

  Теги заголовков не должны использоваться в качестве стилей.

11. Контент

- **Стили для контентных элементов должны быть прописаны: параграфы, изображения, обычные и нумерованные списки, таблицы**

  Это необходимо делать для того, чтобы после разработки сайта, было удобно размещать на нём разнообразный контент, и не приходилось поправлять стили на каждой новой странице. Корректно должны быть прописаны стили для тегов: ul, ol, li, img, a, a:hover, a:focus, p, h1-h6, table, td, th, strong, em, i, b.

- **Блоки сайта не должны расползаться при слишком больших размерах содержимого этих блоков**

  Например, такое может происходить, если в блок с небольшой шириной поместить видео с youtube, слишком большую картинку, или таблицу с большим количеством колонок.

- **Необходимо тестировать сайт с реалистичными изображениями и текстами**

  Если сайт на русском языке, то тестовые тексты должны быть на русском. Тексты и изображения в идеале должны соответствовать тематике сайта. Цветовая гамма изображений также должна подходить, например, если на сайте используются только черно-белые изображения, то такие же должны быть и в верстке макетов.

- **Блоки должны корректно отображаться при любом количестве контента внутри них**

  В том числе, и с пустым содержимым.

- **Проверка орфографии, в том числе и в самом интерфейсе**
- **Наличие clearfix у контейнеров с элементами со стилями float: left и float: right**

  Если такого нет, то при изменении контента, верстка может поломаться.

- **Некорректная микроразметка**

Для определения корректности микроразметки нужно использовать специальные сервисы для проверки. Яндекс.Вебмастер умеет проверять несколько популярных форматов микроразметки, рекомендуется использовать его для проверки.

- **Корректное отображение валюты**

  Все знаки валюты должны быть одинаковыми на сайте, а также отмечены специальным символом, например, символом рубля — ₽.

- **Проверка вместимости длинных названий**

  Названия не всегда могут помещаться в отведенное для них место, в таком случае нужно или обрезать, или настроить стили так, чтобы вёрстка не ломалась при слишком длинных названиях. Тоже самое относится к элементам с ценами, на боевом сайте цены могут быть длиннее, чем на макетах. Цены и другие числа не должны переносится на новую строку.

- **Различная высота элементов в слайдерах**

  Например, слайдеры карточек товаров могут содержать элементы различной высоты. Если эти элементы имеют фон или границы, то необходимо выстраивать им одну высоту, чтобы смотрелось аккуратнее.

12. Изображения

- **Корректное распределение файлов изображений**

  Изображения, не являющиеся интерфейсом сайта, должны лежать отдельно от файлов интерфейса.

- **Корректное подключение изображений**

  Изображения, не являющиеся интерфейсом, по возможности нужно делать через тег img, а не через блок с style="background-image", если это не фон.

- **Наличие файла спрайтов для изображений интерфейса**

  В идеале, все изображения в интерфейсе должны быть в одном svg-спрайте.

- **Корректное подключение файла спрайтов**

  Спрайты должны находится в отдельном файле и подключаться с помощью тега use. Кроме того, файл спрайтов должен корректно подключаться на старых браузерах. Например, для Internet Explorer необходим дополнительный скрипт для корректного подключения svg-файлов svg4everybody.

- **Корректное центрирование изображений в контейнерах**

  Если изображение находится внутри контейнера с фиксированными размерами, то его нужно центрировать с помощью flex.

- **Некорректное содержимое svg-файлов**

  Svg-файлы не должны содержать в себе лишние теги, которые не отображаются в интерфейсе и тормозят загрузку. Svg-файлы должны быть минифицированы.

- **Корректные размеры изображений**

  Размеры изображений должны точно совпадать с размерами контейнера, в котором они расположены или должны быть увеличены в 2-3 раза для поддержки мониторов retina.

- **Корректные стили для изображений**

  Если изображения загружаются пользователями, то нельзя допускать того, чтобы изображение могло исказиться в пропорциях с помощью стилей.

- **Адаптация изображений под мониторы высокого разрешения**

  Под retina-мониторы физический размер изображений должен быть в два а в идеале в 3 раза больше, в зависимости от требований проекта.

- **Плохая оптимизация медиа-файлов**

  Все медиа-файлы (изображения, видео) должны быть максимально сжаты для оптимального соотношения размер/качество и должны воспроизводится на всех поддерживаемых браузерах.

  Чтобы понять, достаточно ли сжаты изображения на сайте, достаточно проверить эту страницу через сервис Google PageSpeed Insights. Этот сервис укажет, какие изображения недостаточно оптимизированы, и напишет рекомендации, что надо сделать в этом случае. Для пользовательских изображений можно использовать сторонний сервис для сжатия, например https://tinypng.com/ или аналогичный. Для остальных типов медиа файлов нельзя указать конкретные рекомендации, в целом нужно пытаться как можно сильнее уменьшить размеры файлов.

- **Отсутствие ленивой загрузки изображений**

  Все тяжелые файлы (изображения и видео) должны загружаться отложено, после всего остального контента, а в идеале после того, как пользователь прокрутил до места, где эти файлы находятся.

- **Изображения разных пропорций, загруженных в одной галерее**

  В блоках, где находится много изображений, все эти изображения должны быть одних пропорций.

- **Наличие оптимизации изображений, загружаемых пользователями**

  Все изображения, загружаемые через CMS, должны быть оптимизированы, а также их размер должен подгоняться под блок, где они находятся.

- **Изображение капчи должно центрироваться относительно соседних блоков**

  Изображения капчи обычно подгружаются со сторонних сервисов, и нельзя полностью настроить их размер, поэтому изображение должно центрироваться по вертикали относительно соседних блоков.

13. Ссылки и кнопки

- **Наличие в ссылках на сторонние сайты атрибута rel="noopener"**

  Этот атрибут необходимо добавлять в целях безопасности, чтобы через сторонние сайты нельзя было получить доступ к основному сайту.

- **Выделение интерактивных элементов при наведении и фокусе**

  Часто это не показано на макетах сайта и не обговаривается отдельно, но для удобства использования сайта необходимо выделять интерактивные элементы на сайте. Также ссылки должны корректно подсвечиваться при выделении их через tab. Требуется обсуждение с дизайнером.

- **Телефоны и электронные почты должны быть прописаны как ссылки**

  В обратном случае, либо пользователю придется вручную копировать телефон или почту, либо устройство само выделит телефон ссылкой, что может нарушить стили сайта.

- **Все нажимаемые области должны иметь cursor: pointer**

  Это позволяет пользователю показать, какие элементы на сайте являются интерактивными.

- **Наличие атрибутов target="_blank"**

  Данный атрибут указывает, чтобы ссылка открывалась в новом табе. Её необходимо проставлять на ссылках на сторонние сайты, ссылках на почтовые адреса и на ссылки, находящиеся в формах, для того чтобы пользователь не потерял заполненные поля формы.

- **Наличие корректных атрибутов type и role для кнопок и ссылок**

  Атрибут type не валиден на тегах <a>, атрибут role="button" нельзя ставить для ссылок, по которым можно перейти на другую страницу. Часто от этого могут сломаться стили или скрипты.

- **Наличие специальных общих классов для кнопок и полей ввода**

  Имеются ввиду общие классы типа btn или form-control для того, чтобы можно было применять общие стили для кнопок или полей ввода, а не застиливать каждую кнопку отдельно.

- **Стилизация кнопок, полей ввода и чекбоксов без помощи скриптов**

  Разработка стилей элементов интерфейса через скрипты в любом случае утяжеляет сайт, делает его менее удобным в разработке и может привести к ошибкам, так что к этому приходить надо как можно меньше.

- **Наличие пустых ссылок**

  На сайте не должно быть ссылок, которые никуда не ведут, например ссылки с href="#". Если такой элемент всё же необходим, то нужно использовать button для этого. В крайнем случае, нужно писать href="javascript:void(0)", чтобы при нажатии на эту ссылку пользователя не прокручивало к началу страницы. Наличие пустых ссылок можно проверить разными программами, мы пользуемся Chrome-расширением Check My Links.

- **Ссылки, ведущие на небезопасные ресурсы**

  На защищенном сайте (https) не должно быть ссылок на ресурсы, доступные по небезопасному соединению (http). В таком случае весь сайт перестает считаться безопасным.

- **Некорректное поведение кнопок на тач-устройствах**

  На тач-устройствах нет такого понятия как наведение, поэтому все подобные взаимодействия на тач-устройствах должны происходить при нажатии на элемент.

- **Незастиленные ссылки при наведении**

  Часто ссылки стилизуют как в макете, но забывают убрать стандартные стили при наведении. Из-за этого, при наведении, ссылка подчеркивается, что не всегда выглядит презентабельно.

- **Незастиленные ссылки при фокусе**

  Помимо того, что ссылки необходимо выделять при фокусировании, эти стили должны сочетаться с другими стилями сайта.

- **Наличие неактивных кнопок, на которые можно нажать**

  Неактивные элементы должны быть полностью неактивны. На них не должно быть курсора «пальца», и также нужно отключать скрипты, срабатывающие при нажатии этих кнопок.

14. Футер

- **Расположение футера, если контента меньше, чем на всю высоту экрана**

  Под футером не должно быть пустого пространства.

- **Наличие фиксированной высоты у футера**
- **Отступы перед футером должны быть одинаковые на всех страницах**
- **Слишком маленький отступ внизу футера**

  Рекомендуется делать не слишком маленький отступ под футером, чтобы на узких экранах при нажатии на элементы в футер случайно не нажать на кнопки на самом телефоне.

15. Формы

- **Правильно прописанные заголовки полей (label)**

  Позволяет пользователю проще ориентироваться при заполнении форм. Кроме того, все заголовки полей должны быть слинкованы с самыми полями через атрибут for.

- **Проверка стилей кнопок и полей ввода**

  Все кнопки и поля ввода должны быть в одном стиле, рядом стоящие элементы не должны расходится по высоте и должны иметь схожие отступы, скругления и шрифты. Если на сайте элементы форм везде имеют разный вид, это выглядит неаккуратно, и может испортить впечатление от сайта. Кроме того, сайт становится потом проще дорабатывать, т.к. разработчик будет знать, что все кнопки и поля ввода идентичны и их не нужно каждый раз оформлять заново.

- **Наличие атрибутов для ограничения длины ввода**
- **Наличие масок для полей ввода**

  Как минимум маски можно поставить на все цифровые поля: телефоны, цены, даты.

- **Клиентская валидация полей ввода**

  Даже, если поле не относится к какой-либо форме, его тоже нужно валидировать прямо во фронтенде и сразу показывать ошибку, если что-то введено неправильно. Необходимо валидировать поля ввода телефона, электронной почты, даты, цены, числовые поля. Валидация всех подобных полей должна быть одинаковая на всем сайте.

- **Корректные атрибуты type у полей ввода**

  Например, для телефонов нужно указывать type="tel", для email-ов — type="email" и т.п. В мобильных браузерах это позволяет открывать правильную клавиатуру при фокусе на поле.

- **Корректная работа с русскими и английскими текстами при заполнении. Многие плагины для валидации или масок некорректно работают с русскими символами, так что на это нужно обращать внимание при использовании js-плагинов**
- **Изменение размеров textarea не должно ломать верстку**

  Обычно, для этого достаточно запретить изменение размера поля по ширине.

- **Различные стили элементов форм в разных браузерах и устройствах**

  Для одинакового вида элементов во всех браузерах необходимо сбрасывать стили для элементов форм.

- **Наличие уведомления после отправления формы**

  Уведомление может быть в виде попапа или в виде выделенного текста над формой, главное, чтобы это уведомление было.

- **Корректный вид уведомлений**

  Уведомления об отправке форм должны быть в стиле сайта, для этого не должен использоваться стандартный alert.

- **Наличие клиентской валидации**

  Должна обязательно быть серверная валидация, проверяющая правильность заполнения формы при отправке этой формы, но клиентская валидация срабатывает мгновенно при взаимодействии пользователя с формой. Улучшает отзывчивость сайта.

- **Корректный сброс формы после отправки**

  Все поля должны очищаться, попапы — закрываться, все классы валидации должны быть сброшены.

- **Проверка отправки формы по нажатию Enter**

  Деактивация кнопки отправки не гарантирует того, что форму нельзя отправить.

- **Корректная работа форм при нажатию кнопки «Назад» в браузере**

  При возвращении на страницу с заполненной формой поля формы остаются заполненными, но скрипты для этих форм не воспроизводятся.

- **Некорректная повторная отправка форм**

  Часто после отправки формы воспроизводятся скрипты, но некорректно сбрасываются после этого. Все формы нужно отправлять повторно.

- **Слишком высокие формы в мобильных браузерах**

  В мобильных браузерах формы растягиваются вертикально. После этого многие формы становятся слишком длинными, особенно те, где одна часть формы меняет другую. Необходимо стараться уменьшать формы по вертикали.

hello@100up.ru
© 2023, 100UP